

Multi Agent Reinforcement Learning Based Swarm Navigation

Omer Yigit

*Dept. of Mechatronics Engineering
Istanbul Bilgi University
Istanbul, Turkey
omer.yigit@bilgi.edu.net*

Suleyman Efe Ceylan

*Dept. of Mechatronics Engineering
Istanbul Bilgi University
Istanbul, Turkey
efe.ceylan@bilgi.edu.net*

Sevval Selin Palas

*Dept. of Mechatronics Engineering
Istanbul Bilgi University
Istanbul, Turkey
selin.palas@bilgi.edu.net*

Arda Isik

*Dept. of Mechatronics Engineering
Istanbul Bilgi University
Istanbul, Turkey
arda.isik02@bilgi.edu.net*

Emir Bekar

*Dept. of Mechatronics Engineering
Istanbul Bilgi University
Istanbul, Turkey
emir.bekar@bilgi.edu.net*

Berkay Vatansever

*Dept. of Mechatronics Engineering
Istanbul Bilgi University
Istanbul, Turkey
berkay.vatansever@bilgi.edu.net*

Yesim Oniz

*Dept. of Mechatronics Engineering
Istanbul Bilgi University
Istanbul, Turkey
yesim.oniz@bilgi.edu.tr*

Abstract—Swarm navigation in dynamic and complex environments represents a critical challenge for both decentralized control systems and dynamic obstacle avoidance. Therefore, this study presents a robust, decentralized Multi Agent Reinforcement Learning (MARL) model which uses a distributed-proximal policy optimization (PPO) algorithm and enables the navigation of swarms collectively in a decentralized manner, using a shared policy without explicit communication among agents. In order to train each agent to learn how to transform an 8 dimensional vector into a specific action from their perspective in a two dimensional arena (thus enabling them to take action in the arena), we represent the system as a decentralized Multi Agent Markov Decision Process (MAMDP) and use a multi component reward function that provides penalty against collisions, rewards agents for successful progress toward the goals, and dampens velocity to promote stability. Our simulation results show quick convergence during training, and our agents achieve high success rates in all randomized configurations of the environment. Furthermore, our agents are able to generalize well across new, unseen obstacles; therefore, they are able to maintain their ability to navigate efficiently and safely in previously unseen obstacle configurations.

Index Terms—Multi Agent Reinforcement Learning, Swarm Robotics, Decentralized Control, Proximal Policy Optimization

I. INTRODUCTION

Swarm intelligence is a major part of AI and is becoming much more significant, as many complex problems require solutions to be found in a reasonable timeframe, and less than perfect solutions can be acceptable. Swarm intelligence primarily uses biological references to model the collective behaviors of large numbers of highly organized animals search-

ing for survival [1]. In the control phase, learning algorithms are utilized to regulate our swarms, and learning algorithms are represented as one of the most significant components of the swarm intelligence concept. Swarms typically operate in two primary modes: decentralized or centralized. One of the most critical distinguishing characteristics of the two modes of operation is that a leader is not present in the decentralized mode of operation; however, in the case of the centralized mode of operation, a single leader is typically present [2], [3]. Swarm robotics provides us with the ability to view the mechanical environments of swarms in nature. Swarm robotics is focused upon studying and understanding the behavior of multiple agents (robots) in a simple robot environment. Many of the examples that swarm robotics study occur naturally such as ant colonies, bee swarms and schools of fish. Reliable swarm routing in complex environments represents an ongoing and important problem in the area of swarm intelligence, due to the presence of dynamic obstacles and decentralized control. Reinforcement Learning offers a suitable platform from which to design optimal routing policies through learning rather than programming. Compared to traditional rule based routing methodologies, RL based routing is capable of handling uncertainty in the environment by learning through trial and error, particularly in multi agent decentralized systems. Multi-agent reinforcement learning has attracted increasing amounts of research in recent years, motivated by advancements in both swarm intelligence and reinforcement learning. Based on advances in swarm intelli-

gence and reinforcement learning, multi agent reinforcement learning has attracted more and more researchers aiming to develop a framework of cooperative distributed systems within complex environments [4]. Decentralized MARL systems in which agents learn their own policies using local observations are also scalable and robust to swarm robotics applications, highly realized in MARL surveys [4], [5]. The modeling of our system assumes that individual robots are independent learning agents that interact with a common environment. A common neural policy is provided with observations from individual robots, similar to many cooperative MARL design approaches [6]. The approach to our training pipeline is based on policy gradient reinforcement learning, which is implemented on modern deep learning RL frameworks, which is consistent with recent studies indicating that deep MARL can be applied successfully to swarm navigation and coordination tasks [7]. The training methodology is based upon a policy gradient reinforcement learning approach; furthermore, the same deep learning RL frameworks that have been used in several recent studies demonstrating successful application of deep MARL to swarm navigation and coordination tasks are utilized.

II. METHODOLOGY

A. Environment Formulation

In the direct utilization of the swarm navigation problem, we simulate a completely observed multi agent problem with the MultiAgentEnv [8] RLlib framework and create an environment made up of N decentralized agents that operate within a continuous 2D square arena with dimensions of 10×10 . The agents are all modeled as point-masses with a given radius and move in accordance with a simple set of kinematic rules to act in response to discrete action directions. In each episode, the spawning of the agents occurs randomly in the beginning at detached initial points. There exists a goal point of the world and several circular motionless barriers sampled in the arena.

The characteristics of the observation model of each agent are eight: the position of the agent self, global goals position coordinates, relative location of nearest agent neighboring, and relative positions of nearest obstacle. The formulation provides the agents with local meaningful but non-communicative sensory input, which allows decentralized decision making.

The environment may also impose termination by limits of boundaries, collision limits, and episode termination occurs when an agent reaches a goal or the time step limit is exceeded. Our swarm navigation task was referred to as a decentralized one Multi Agent Markov Decision Process (MAMDP) [6], [9] defined by the pair $M = \langle N, S, O, A, P, r \rangle$, where $N = 1, 2, \dots, n$ denotes the set of agents [7]. Each agent i moves in a continuous 2D arena in $[-5, 5]$ boundaries. The global state at time t is defined as

$$s_t = \{x_t^i\}_{i=1}^N \cup \{g\} \cup \{o_k\}_{k=1}^K, \quad (1)$$

where $x_t^i = (x_t, y_t)$ denotes the position of agent i , g is the global goal position, and each obstacle $o_k = (o_{kx}, o_{ky}, r_k)$ is represented by its center coordinates and radius [4].

Since agents operate without explicit communication, each agent receives a local observation

$$o_t^i = [x_t^i, g, (x_t^{\text{nbr}} - x_t^i), (x_t^{\text{obs}} - x_t^i)], \quad (2)$$

consisting of its own position, the goal coordinates, and the relative positions of (i) the nearest neighboring agent and (ii) the closest obstacle [10]. This results in an 8-dimensional observation vector, consistent with the implemented environment.

The discrete action space is defined as

$$A = \{0 : \text{forward}, 1 : \text{left}, 2 : \text{right}, 3 : \text{idle}\}, \quad (3)$$

where actions deterministically update the agent position according to simple kinematic rules with step size Δt [11]–[13]. The transition dynamics are therefore deterministic and given by

$$x_{t+1}^i = f(x_t^i, a_t^i, g), \quad (4)$$

including boundary clipping and obstacle proximity constraints [11]. The reward function is designed to encourage progress toward the goal while penalizing collisions:

$$r_t^i = 3.0 \Delta d_t^i - 0.1 d_t^i - 10.0 \mathbb{I}_{\text{coll}}^i + 15.0 \mathbb{I}_{\text{goal}}^i - 5.0 \|\mathbf{v}_t^i\| \mathbb{I}_{\text{goal}}^i + 50.0 \mathbb{I}_{\text{team}} \quad (5)$$

Episodes terminate when an agent reaches the goal or when the maximum number of time steps is reached [4], [6]. This formulation captures the operational behavior of the implemented environment while providing a mathematically grounded MAMDP structure suitable for decentralized multi-agent reinforcement learning.

B. Decentralized Multi Agent Reinforcement Learning Setup

We make a complete adoption of a multi-agent reinforcement learning (MARL) configuration which is completely decentralized. In this configuration, every agent chooses its actions acting separately on the basis of observations alone made by it locally, receiving individual rewards. Agents are regarded as interacting independent learners with a shared environment. Even though there are more than one agents at the same time, learning of policy is done in an inaccessible manner to the observations or actions of other agents; a setting usually so-called independent multi agent learning [14].

To encourage the behavior of a homogeneous swarm and enhance scalability, all agents have a common policy network with identical parameters. Decentralization is maintained in policy sharing implementation, and at the same time, permitting experience acquired by others to be extensively practiced by the agents to contribute to one policy update learned in swarm learning and decentralized MARL [6].

Although complete decentralization of learning can bring about non-stationarity due to simultaneous policy changes, this choice of design is deliberate; since it allows scalable and communication-free deployment in real-world multi robot systems [9].

C. Proximal Policy Optimization For Swarm Coordination

Proximal Policy Optimization (PPO) is employed as the underlying learning algorithm for decentralized swarm coordination due to its favorable convergence properties and empirical robustness in policy gradient-based reinforcement learning [15]. In the considered setting, each agent is treated as an independent decision-making entity that learns a policy solely from its own local observations and received rewards.

The learning framework follows a fully decentralized multi-agent reinforcement learning paradigm, in which neither centralized critics nor joint state information are utilized during training. Each agent maintains its own policy and value function, and learning is performed independently using locally collected trajectories. This formulation corresponds to the independent learner assumption originally introduced in early multi-agent reinforcement learning studies [14] and later formalized in comprehensive MARL surveys [4]. While such decentralized learning induces non-stationarity from the perspective of individual agents, policy gradient methods with trust-region-inspired constraints, such as PPO, have been shown to exhibit stable behavior in practice [15].

Formally, each agent i learns a stochastic policy

$$\pi_{\theta_i}(a_t^i | o_t^i), \quad (6)$$

parameterized by θ_i , which maps its local observation o_t^i to a discrete action a_t^i . Policy optimization is performed by maximizing the clipped PPO surrogate objective

$$L^{\text{PPO}}(\theta_i) = \mathbb{E}_t \left[\min \left(r_t(\theta_i) \hat{A}_t^i, \text{clip}(r_t(\theta_i), 1 - \epsilon, 1 + \epsilon) \hat{A}_t^i \right) \right] \quad (7)$$

where $r_t(\theta_i)$ denotes the probability ratio between the updated and previous policies, $\hat{A}_t^i$ is an advantage estimate computed from locally collected trajectories, and ϵ is the clipping parameter that constrains the policy update.

Although agents are trained independently, all agents share the same policy architecture and hyperparameter configuration, differing only in their network parameters and experienced trajectories. This homogeneous-agent assumption facilitates scalability and enables the emergence of coordinated swarm behavior without explicit communication or centralized supervision. Similar decentralized PPO-based formulations have been successfully applied to multi-robot navigation and collision avoidance tasks under comparable assumptions [11], [13].

D. Training Pipeline And Implementation Details

The training pipeline is a fully decentralized multi-agent reinforcement learning paradigm, in this way all agents learn within the environment as they interact with it. Training is conducted independently and is based on locally collected experience only. There is no central critic, replay buffer, or joint state information utilized during training, ensuring strict decentralization throughout the learning process.

At each training iteration, all agents execute their current policies π_{θ_i} in parallel and collect trajectories consisting

of local observations, actions, and rewards. Each trajectory segment is stored locally and used to compute advantage estimates $\hat{A}_t^i$ based on the agent's own value function predictions. This decentralized data collection aligns with the independent learner assumption originally introduced in early MARL literature [14] and later formalized in comprehensive surveys [9]. The Proximal Policy Optimization algorithm [15] is used to carry out policy updates. For each agent, the clipped stochastic gradient is used in optimization of PPO surrogate objective over several epochs/minibatches sampled of the local obtained trajectory. This on-policy update scheme reduces overly big policy changes and is a contributor to learning behavior in non-stationary multi agent systems which remains stable.

The neural network architecture is the same in all the agents and hyperparameter setting, a factor of a homogeneous swarm robots usually use agent setting [6]. Frequent updates of parameters are however done independently, allowing agents to make their policies flexible according to personalities experiences as well as being scalable with regard to the number of agents.

This is carried out in an iterative process by training the process until convergence or a specified number of steps in interacting with the environment has been achieved. This decentralized training pipeline allows the development of coordination of swarm behavior in the absence of explicit communication and centralized supervision and has been demonstrated to work in multi robot navigation and collision avoidance tasks. [11], [12].

III. SIMULATION

A. Experimental Environment Setup

The swarm navigation task is evaluated in a continuous two-dimensional simulation environment with a square arena of size 10×10 . The environment contains multiple static circular obstacles and a single global goal region. At the beginning of each episode, obstacle locations, agent initial positions, and the goal position are randomly sampled to ensure robustness against environmental variations. Each agent is modeled using an acceleration-velocity dynamic model with a fixed integration time step of $\Delta t = 0.1$, resulting in a maximum episode duration of 30 seconds (300 simulation steps). Agent velocities are bounded by a maximum speed of 1.5 to ensure physically plausible and numerically stable motion. Agents are represented as circular bodies with a fixed radius, and collisions with obstacles or arena boundaries are explicitly detected. The global objective function is represented as a circular goal area to prevent agents from moving back and forth towards the goal when they are close to the goal area by applying velocity damping, which encourages agents to converge smoothly to the goal, and remain stationary at the time of arrival. An episode is terminated early if all agents have reached the goal area. Otherwise, an episode will terminate

after a predetermined number of steps (maximum step limit) have been completed.

B. Observation and Action Spaces

Agents receive their continuous-valued local observation vectors which include their own states and goal related data. Each local observation vectors are made up of an agent's location, velocity, absolute goal location and the relative distance from the agent to the goal creating an eight dimensional observation space. The two dimensional action space is continuous with the agent being able to accelerate at any rate within a specified range. In addition the actions are scaled so that there are gradual changes in the agents velocity. The use of this formulation provides for the ability to generate smooth paths and also allows for the agents to be able to make dynamic adjustments based on feedback from their environment.

C. Reward Design

The reward function has been developed to encourage agents to navigate in a safe manner toward goals (efficiently), and to avoid behaviors that are unsafe. Agents at each time step of the simulation will receive a negative reward for each unit of Euclidean distance from the target location. This will encourage the agent to continue moving toward the target location. Large negative penalties will be received by the agents for collisions with obstacles, or the walls of the arena. A positive terminal reward will be given when the agent is within the target region. This design of the reward function will balance the need for exploration, safety, and efficient navigation for all agents under decentralized control. The Reward Function has multiple components to support both efficient navigation and completing tasks, as well as promoting safe agent cooperation.

The Reward Function contains multiple components, each designed to promote both efficient navigation and task accomplishment while ensuring safe cooperation among agents. A summary of the Reward Structure and Components can be found in Table I.

TABLE I
REWARD COMPONENTS USED IN THE SWARM NAVIGATION ENVIRONMENT

Reward Component	Value
Progress Reward	$3.0 \cdot \Delta d$
Distance Penalty	$-0.1 \cdot d$
Obstacle / Boundary Collision	-10.0
Goal Staying Reward	+15.0
Velocity Penalty at Goal	$-5.0 \cdot \ \mathbf{v}\ $
Team Success Bonus	+50.0

D. MARL Training Configuration

The learning problem is formulated as a decentralized multi-agent reinforcement learning task following the centralized training and decentralized execution paradigm. All agents share a common policy network through parameter sharing

while selecting actions independently based on local observations. Proximal Policy Optimization (PPO) is employed as the learning algorithm due to its stability in continuous control and multi-agent settings. Training is performed using a discount factor $\gamma = 0.99$ and a learning rate of 1×10^{-4} . Experience is collected in parallel using multiple rollout workers to improve data efficiency and training stability.

E. Implementation Details

The suggested framework was developed using the Ray RLlib library with a PyTorch backend for implementation. Each batch (i.e., each time step of data to be used during optimization) has a size of 4000 "environment steps" and uses stochastic gradient descent for optimization with an initial mini-batch size of 256 and a total of 10 epoches as part of each training session. As the agents are all utilizing a single "shared policy network", this approach allows for scalable training and decentralized agent execution. All experiments were performed on a single workstation that had a graphics processing unit (GPU). At every training session, evaluation metrics are collected by executing test episodes within randomly generated independent environments to ensure that reported results indicate both the improvements made to the policy and robustness.

IV. RESULTS

A. Learning Convergence and Task Success

The learning evolution of the proposed multi-agent reinforcement learning model was assessed by the mean-episode reward, and success rates; the mean-episode reward and the related success rates are shown in Fig.1 as a function of training iteration. As can be seen from the mean-episode reward in figure three, the overall trend is one of increasing mean-episode reward, however, this trend contains significant random fluctuations which may be attributed to the randomization of both the agent's initial position, and the configuration of obstacles within the environment. These fluctuations indicate that although there were some improvements in the agent's policy (as indicated by the mean-episode reward), it has also demonstrated an ability to remain effective even with environmental variability. In addition to this, the success rate of the agents demonstrates an almost immediate increase in success rate for the first few training iterations, followed by convergence towards nearly perfect performance.

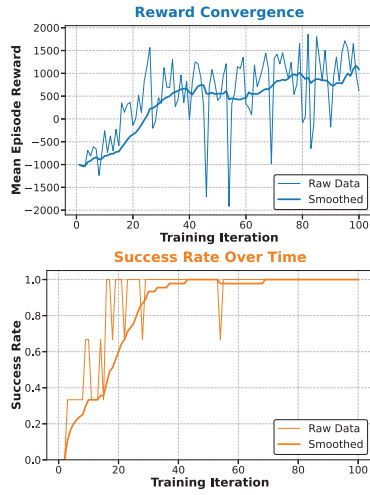


Fig. 1. Learning performance during training mean episode reward convergence and success rate over training iterations.

Furthermore, Fig.2 presents a representative successful episode executed after training. The snapshot illustrates the trajectories of all agents as they navigate toward their shared goal in a randomized environment configuration at the end of the 300 steps.

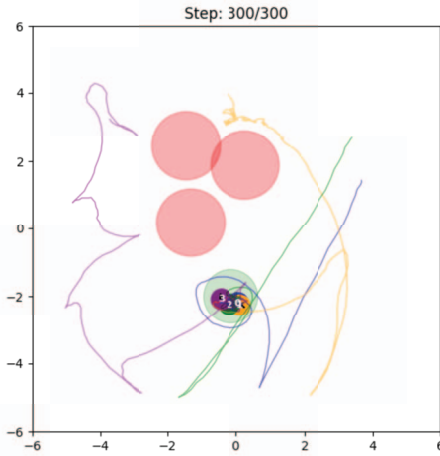


Fig. 2. Representative successful episode after training. The trajectories of all agents are shown as they navigate around obstacles and converge to the shared goal in a randomized environment.

B. Navigation Efficiency and Safety

Time to reach goals (Navigation Efficiency), and the number of collisions (Safety) were assessed during the study using two variables: mean time to the goal, and collision frequency. From Fig.3 we can see that the agents mean time to the goal steadily decreased as they trained which indicates that the agents gradually became more effective in their obstacle avoidance strategies and improved their cooperation with each other. Initially the agents experienced a large increase in collision frequency while exploring but the agents collision

frequency remained stable at or around this high level for most of the remaining training sessions indicating that the agents had learned to weigh the positive effects of rewards from exploratory behaviors against the negative consequences of collisions and therefore had learned to navigate safely.

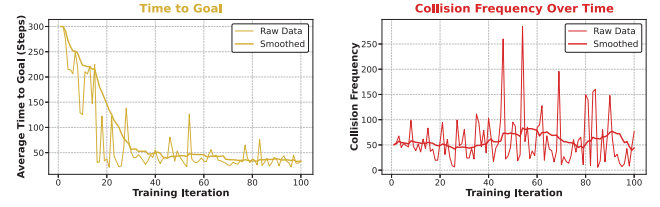


Fig. 3. Navigation efficiency metrics during training: average time to reach the goal and collision frequency over training iterations.

C. Motion Regulation and Control Stability

The smoothness and consistency of an agent's movement during its interaction with other agents can be determined by examining how the average speed of all agents changes as they are trained. The illustration in Fig.4 shows the average speed of the agents over time (iterations) as training progresses. As training is completed, the average speed of the agents decreases gradually. Especially when agents approach the goal location. This decrease in average speed of the agents is indicative of the agents learning to slow down (decelerate) when approaching their goals rather than displaying sudden or oscillating movements. This type of behavior provides evidence for the effectiveness of the velocity based rewards included in the rewards structure used to train the agents in achieving stable and controlled swarm navigation.



Fig. 4. Agent speed over training iterations, illustrating increasingly smooth and controlled motion behavior.

V. CONCLUSION

This paper describes an multi agent reinforcement learning framework for swarm navigation through obstacles in complex environments. A shared policy PPO approach along with a carefully developed reward structure allows the proposed method to enable multiple agents to collaborate as they all move towards a common objective, while avoiding collisions.

The experimental results show rapid training convergence and high rates of success, lower times to achieve objectives and smoother movements of each agent in the swarm across the variety of randomly generated environments. Further, trajectory visualizations illustrate the learned policies are able to generalize to previously unobserved configuration

of obstacles; resulting in a coordinated and efficient swarm behavior.

Comparing the proposed learning based approach with random policies demonstrates its relative effectiveness. Future work is planned to extend this framework to include heterogeneous agents, dynamic obstacles and also to investigate the effect of decentralized communication limitations. Also, applying the learned policies in simulation to robotic platforms in real world applications may provide a promising direction for future swarm robotics research and development.

REFERENCES

- [1] A. Chakraborty and A. K. Kar, "Swarm intelligence: A review of algorithms," *Nature-inspired computing and optimization*, pp. 475–494, 2017.
- [2] Y. Alqudsi and M. Makaraci, "Uav swarms: research, challenges, and future directions," *Journal of Engineering and Applied Science*, vol. 72, no. 1, p. 12, 2025.
- [3] L. V. Nguyen, "Swarm intelligence-based multi-robotics: a comprehensive review," *AppliedMath*, vol. 4, no. 4, pp. 1192–1210, 2024.
- [4] L. Busoniu, R. Babuska, and B. De Schutter, "Multi-agent reinforcement learning: A survey," in *2006 9th international conference on control, automation, robotics and vision*. IEEE, 2006, pp. 1–6.
- [5] M. Brambilla, E. Ferrante, M. Birattari, and M. Dorigo, "Swarm robotics: a review from the swarm engineering perspective," *Swarm Intelligence*, vol. 7, no. 1, pp. 1–41, 2013.
- [6] K. Zhang, Z. Yang, and T. Başar, "Multi-agent reinforcement learning: A selective overview of theories and algorithms," *Handbook of reinforcement learning and control*, pp. 321–384, 2021.
- [7] F. A. Oliehoek, C. Amato *et al.*, *A concise introduction to decentralized POMDPs*. Springer, 2016, vol. 1.
- [8] E. Liang, R. Liaw, R. Nishihara, P. Moritz, R. Fox, K. Goldberg, J. E. Gonzalez, M. I. Jordan, and I. Stoica, "RLlib: Abstractions for distributed reinforcement learning," in *International Conference on Machine Learning (ICML)*, 2018. [Online]. Available: <https://arxiv.org/pdf/1712.09381>
- [9] L. Busoniu, R. Babuska, and B. De Schutter, "A comprehensive survey of multiagent reinforcement learning," *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, vol. 38, no. 2, pp. 156–172, 2008.
- [10] R. Lowe, Y. I. Wu, A. Tamar, J. Harb, O. Pieter Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," *Advances in neural information processing systems*, vol. 30, 2017.
- [11] M. Everett, Y. F. Chen, and J. P. How, "Motion planning among dynamic, decision-making agents with deep reinforcement learning," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 3052–3059.
- [12] J. Lin, X. Yang, P. Zheng, and H. Cheng, "End-to-end decentralized multi-robot navigation in unknown complex environments via deep reinforcement learning," in *2019 IEEE international conference on mechatronics and automation (ICMA)*. IEEE, 2019, pp. 2493–2500.
- [13] —, "Connectivity guaranteed multi-robot navigation via deep reinforcement learning," in *Conference on Robot Learning*. PMLR, 2020, pp. 661–670.
- [14] M. Tan, "Multi-agent reinforcement learning: Independent vs. cooperative agents," in *Proceedings of the tenth international conference on machine learning*, 1993, pp. 330–337.
- [15] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.